

UNIVERZITET U BEOGRADU – ELEKTROTEHNIČKI
FAKULTET
KATEDRA ZA ELEKTRONIKU

DIGITALNA ELEKTRONIKA 1

Materijali za računske vežbe

**SINTEZA KOMBINACIONIH MREŽA POMOĆU KOLA
NISKOG STEPENA INTEGRACIJE I STATIČKI HAZARDI**

Vežbe 1

Autori:

Nikola Petrović p.z.nikola@etf.bg.ac.rs
Haris Turkmanović haris@etf.bg.ac.rs

Beograd, 9. oktobar 2026.

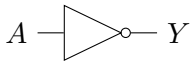
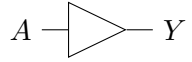
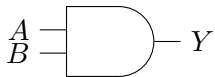
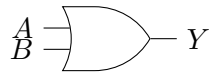
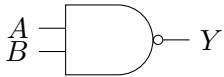
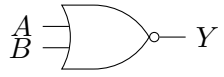
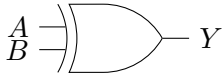
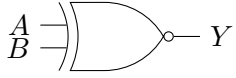
Sadržaj

1	Uvod	2
2	Zadaci	4
2.1	Zadatak 1	4
2.2	Zadatak 2	11
2.3	Zadatak 3	14
2.4	Zadatak 4	17
2.5	Zadatak 5	20
2.6	Zadatak 6	24
3	Zadaci za samostalni rad	27
3.1	Zadatak 1	27
3.2	Zadatak 2	27

1 Uvod

U okviru prvog dela ovih računskih vežbi obrađivaćemo praktične primere vezane za sintezu kombinacionih mreža korišćenjem kola niskog stepena integracije. Pod kolima niskog stepena integracije, za potrebe ovog kursa, podrazumevaćemo grupu osnovnih logičkih kola predstavljenih u tabeli 1.

Tabela 1: Pregled osnovnih logičkih kola

Naziv	Simbol	Funkcija	Tabela istinitosti															
Invertor (NOT)		$Y = \neg A = \bar{A}$	<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Y	0	1	1	0									
A	Y																	
0	1																	
1	0																	
Bafer (BUFFER)		$Y = A$	<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	Y	0	0	1	1									
A	Y																	
0	0																	
1	1																	
I (konjunkcija, AND)		$Y = A \wedge B = A \cdot B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	0	1	0	0	1	1	1
A	B	Y																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
ILI (disjunkcija, OR)		$Y = A \vee B = A + B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	1
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NI (NAND)		$Y = \neg(A \wedge B) = \overline{A \cdot B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	1	1	0	1	1	1	0
A	B	Y																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NILI (NOR)		$Y = \neg(A \vee B) = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	0
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Ekskluzivno ILI (XOR)		$Y = A \oplus B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	0
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Ekvivalencija (XNOR)		$Y = \neg(A \oplus B) = \overline{A \oplus B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	1
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Jedan od bitnijih koraka u procesu sinteze kombinacionih mreža predstavlja proces minimizacije logičkih funkcija. Iako su razvijene različite metode koje imaju za cilj da minimizuju broj logičkih kola potrebnih za sintezu neke logičke funkcije, za potrebe računskih vežbi iz ovog kursa u svrhu minimizacije logičkih funkcija koristićemo metodu baziranu na Karnoovim kartama. Zbog toga, za potpuno razumevanje primera koji slede, neophodno je dobro poznavanje ove oblasti koja je odrađena na časovima predavanja.

Pored poznavanja funkcionalnosti osnovnih logičkih kola, kao i poznavanja metoda za minimizaciju logičkih funkcija, u toku izrade zadataka potrebno je poznavanje određenih algebarskih pravila koja važe u slučaju Bulove algebre. U tabeli 2 predstavljena su neka od osnovnih pravila Bulove algebre koja će biti korišćena kroz izradu zadataka.

Tabela 2: Neka od osnovnih pravila Bulove algebre

Broj pravila	I forma	II forma
1	$0 \cdot A = 0$	$0 + A = A$
2	$1 \cdot A = A$	$1 + A = 1$
3	$A \cdot A = A$	$A + A = A$
4	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
5	$A \cdot B = B \cdot A$	$A + B = B + A$
6	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
7	$A + B \cdot C = (A + B) \cdot (A + C)$	$A \cdot (B + C) = A \cdot B + A \cdot C$
8	$A \cdot (A + B) = A$	$A + A \cdot B = A$
9	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$
10	$A = \overline{\bar{A}}$	

U drugom delu vežbi proučavamo statičke hazarde u kombinacionim mrežama: kratkotrajnu lažnu nulu kada izlaz treba da ostane na jedinici (statički hazard 1), odnosno lažnu jedinicu kada treba da ostane na nuli (statički hazard 0). Uzrok su različita kašnjenja rekonvergentnih putanja. Simulacija može da otkrije impuls za ispitane prelaze i kašnjenja; sama simulacija ne uklanja hazard niti odsustvo impulsa u jednom modelu dokazuje odsustvo hazarda. U dvonivojskoj realizaciji ZP/PZ, uz promenu samo jednog ulaza, statički hazard sprečavamo zajedničkim pokrivanjem svakog para susednih jedinica/nula odgovarajućim implikantom/implikatom. Ova garancija ne obuhvata proizvoljne istovremene promene više ulaza.

2 Zadaci

2.1 Zadatak 1

Funkciju

$$Y = (A + C + D) \cdot (\bar{A} + C + \bar{D}) \cdot (B + \bar{C}) \quad (2.1)$$

realizovati u minimalnoj formi:

- Korišćenjem osnovnih logičkih kola sa proizvoljnim brojem ulaza
- Korišćenjem isključivo NI logičkih kola sa proizvoljnim brojem ulaza
- Korišćenjem isključivo NI logičkih kola sa dva ulaza
- Korišćenjem isključivo NILI logičkih kola sa dva ulaza

Rešenje:

a) Pošto se u okviru postavke zadatka traži realizacija u minimalnoj formi neophodno je izvršiti minimizaciju logičke funkcije korišćenjem nekih od dostupnih metoda za minimizaciju. U okviru ovog kursa, koristićemo metodu Karnoovih karti. Karnoova karta omogućava nalaženje minimalnog dvonivojskog pokrivanja u formi ZP ili PZ (najpre broj članova, zatim broj literala). To nije dokaz globalno najmanjeg broja kola u proizvoljnoj višenivojskoj realizaciji ili biblioteci koja sadrži i XOR kola. Pri poređenju realizacija posebno računamo invertore i broj ulaza kola.

$\begin{smallmatrix} C/D \\ A/B \end{smallmatrix}$	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	1	0	1	1
10	1	0	0	0

Slika 1: Sadržaj Karnoove karte za funkciju (2.1)

$\begin{smallmatrix} C/D \\ A/B \end{smallmatrix}$	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	1	0	1	1
10	1	0	0	0

(a) Proizvod zbirova

$\begin{smallmatrix} C/D \\ A/B \end{smallmatrix}$	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	1	0	1	1
10	1	0	0	0

(b) Zbir proizvoda

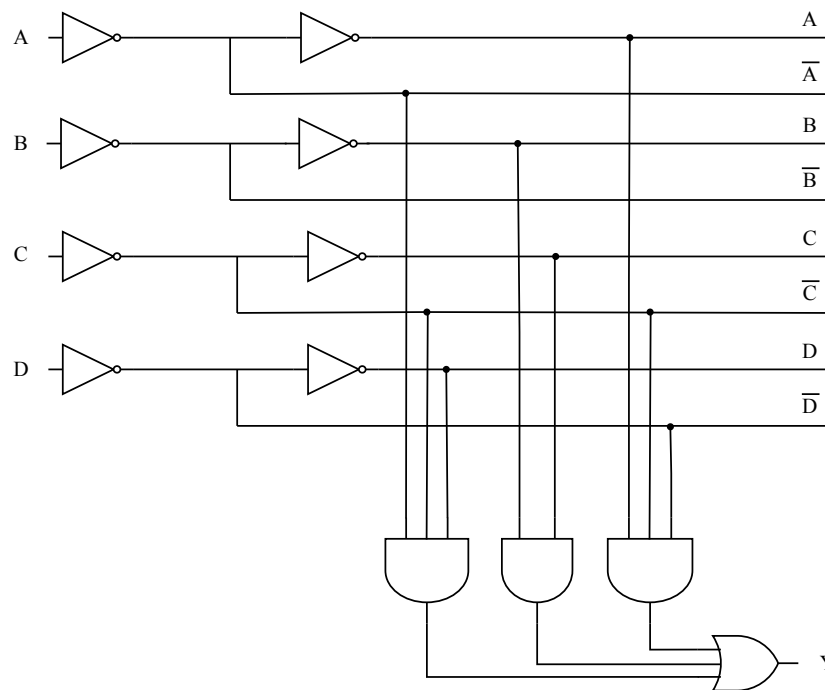
Slika 2: Sadržaj Karnoove karte za funkciju (2.1), gde je a) označena oblast za minimizaciju u formi proizvoda zbirova b) označena oblast za minimizaciju u formi zbira proizvoda

Nakon minimizacije dobijaju se minimalne funkcije u formi zbira proizvoda (2.2) i u formi proizvoda zbirova (2.3). Za realizaciju funkcije (2.2) potrebna su tri I kola i jedno trouglasto ILI kolo; za (2.3) tri ILI kola i jedno trouglasto I kolo. Obe realizacije dodatno zahtevaju tri invertora za A , C i D , ako negacije nisu dostupne: ukupno sedam kola. Na šemama je radi prikaza zajedničkog ulaznog razvoda prikazano po dva invertujuća kola za svaki ulaz: prvo daje komplement, a drugo direktni signal. Taj razvod nije minimalan i ne pripada navedenom broju od sedam kola; prikazana šema ZP sa celim razvodom ima dvanaest kola. Pri minimizaciji hardvera direktni signali se uzimaju sa ulaza, a generišu se samo potrebne negacije. Horizontalni vodovi nose označene signale, a tačke označavaju njihove spojeve sa vertikalnim vodovima; ukrštanje bez tačke nije spoj.

$$Y_{MIN_ZP} = \bar{A}\bar{C}D + A\bar{C}\bar{D} + BC \quad (2.2)$$

$$Y_{MIN_PZ} = (A + C + D)(\bar{A} + C + \bar{D})(B + \bar{C}) \quad (2.3)$$

Na slici 3 je predstavljena realizacija funkcije (2.2).



Slika 3: Implementacija funkcije (2.1) u formi zbira proizvoda.

.....
b) U okviru ove tačke zahtevana je implementacija funkcije (2.1) korišćenjem isključivo NI kola. Sistematski pristup za implementaciju funkcije koja sadrži isključivo NI kola podrazumeva dva koraka:

- 1) Transformaciju minimalne funkcije u formi zbira proizvoda u algebarski zapis koji sadrži isključivo komplementirane proizvode
- 2) Šemu koja sadrži isključivo NI kola a koja se dobija na osnovu algebarskog izraza dobijenog u koraku 1

Korak (1) obuhvata niz transformacija koje za cilj imaju dobijanje algebarskog zapisa u kome isključivo figurišu komplementirani proizvodi. U slučaju da se zahteva realizacija isključivo korišćenjem NI kola sa n ulaza, proizvodi moraju sadržati tačno n činilaca. U okviru ove tačke broj ulaza nije definisan tako da ne postoji ograničenje po pitanju broja dostupnih ulaza u NI

logičko kolo. Kao polazni izraz, nad kojim će se vršiti dalje transformacije, koristi se forma zbira proizvoda funkcije dobijene primenom Karnoove karte. Za slučaj funkcije (2.1) polazni izraz predstavlja (2.2) odnosno zbir proizvoda. Na osnovu pravila Bulove algebre po kome se vrednost funkcije ne menja dvostrukim komplementiranjem iste, dobijamo:

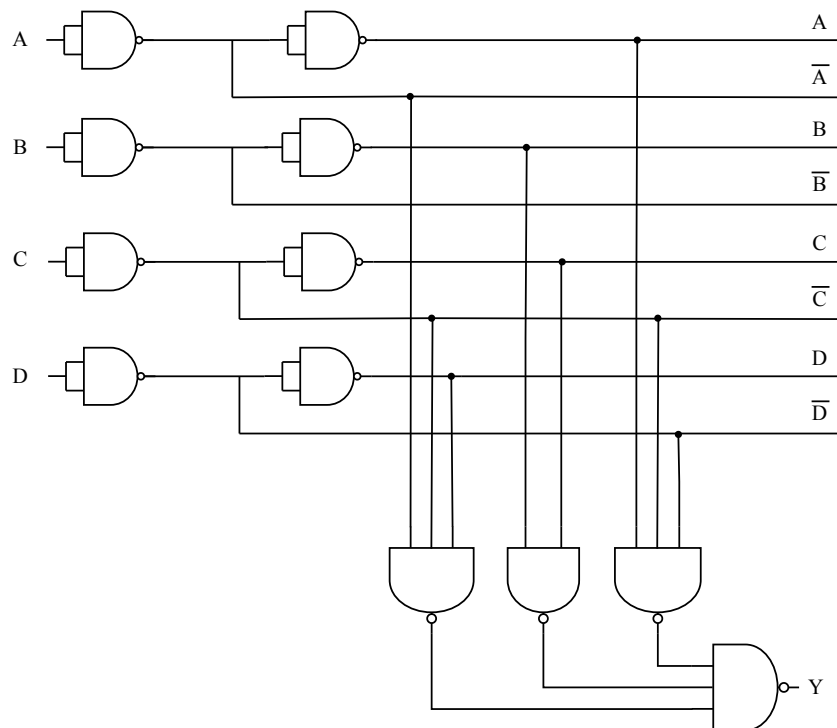
$$Y = \overline{\overline{Y}} = \overline{\overline{\overline{A}CD + \overline{A}C\overline{D} + BC}} \quad (2.4)$$

Daljim transformacijama kao krajnji rezultat dobijamo:

$$Y = \overline{\overline{Y}} = \overline{\overline{\overline{A}CD} \cdot \overline{\overline{A}C\overline{D}} \cdot \overline{BC}} \quad (2.5)$$

Iz funkcije (2.5) možemo uočiti da u njoj figurišu isključivo komplementirani proizvodi, tačnije četiri takva proizvoda (crveni – $\overline{A}CD$; plavi – $\overline{A}C\overline{D}$; zeleni – $\overline{BC}$; i na kraju globalni komplement koji predstavlja celu funkciju).

Na slici 4 je predstavljena implementacija funkcije (2.5).



Slika 4: Implementacija funkcije (2.1) korišćenjem isključivo NI kola sa proizvoljnim brojem ulaza.

Verilog kod: NI realizacija sa slike 4. Svaka dodela predstavlja jedno NI kolo. Na šemi je prikazan potpun ulazni razvod sa osam invertora.

```

module zadatak (
    input logic A, B, C, D,
    output logic Y
);
    logic A_n, B_n, C_n, D_n;
    logic A_p, B_p, C_p, D_p;
    logic I1, I2, I3;
    // Potpun ulazni razvod sa osam invertora, kao na semi.
    assign A_n = ~(A & A);
    assign A_p = ~(A_n & A_n);
    assign B_n = ~(B & B);
    assign B_p = ~(B_n & B_n);
    assign C_n = ~(C & C);
    assign C_p = ~(C_n & C_n);
    assign D_n = ~(D & D);
    assign D_p = ~(D_n & D_n);
    // Tri komplementirana proizvoda sa seme.
    assign I1 = ~(A_n & C_n & D_p);
    assign I2 = ~(A_p & C_n & D_n);
    assign I3 = ~(B_p & C_p);
    assign Y = ~(I1 & I2 & I3);
endmodule

```

Testbench i automatske provere: [tb_zadatak.sv](#).

c) U okviru ove tačke se takođe zahteva realizacija funkcije (2.1) korišćenjem **isključivo dvoulaznih NI** kola. Koraci (1) i (2) su identični kao u prethodnoj tački što podrazumeva da i ovde polazimo od algebarskog zapisa funkcije (2.1) u formi zbira proizvoda (2.2). Nakon niza transformacija (2.6)-(2.9) dobijamo krajnji izraz (2.10).

$$Y = \bar{C}(\bar{A}D + A\bar{D}) + BC \quad (2.6)$$

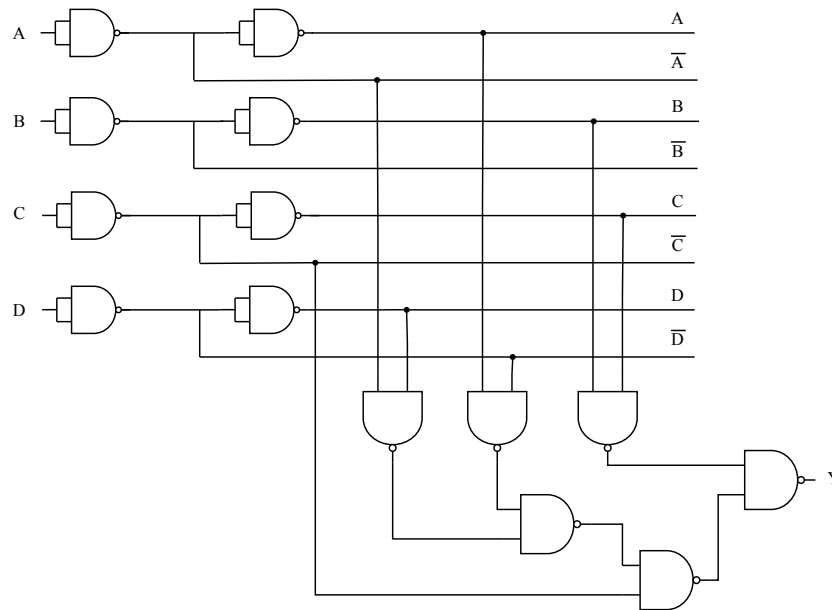
$$Y = \bar{\bar{Y}} = \bar{\bar{C}(\bar{A}D + A\bar{D}) + BC} \quad (2.7)$$

$$Y = \bar{\bar{C}(\bar{A}D + A\bar{D}) \cdot \bar{BC}} \quad (2.8)$$

$$Y = \bar{\bar{\bar{C}(\bar{A}D + A\bar{D}) \cdot \bar{BC}}}} \quad (2.9)$$

$$Y = \bar{\bar{\bar{C}(\bar{A}D \cdot \bar{A}\bar{D}) \cdot \bar{BC}}}} \quad (2.10)$$

Na slici 5 je predstavljena implementacija funkcije (2.10).



Slika 5: Implementacija funkcije (2.1) korišćenjem isključivo dvoulaznih NI kola.

Verilog kod: realizacija isključivo dvoulaznim NI kolima sa slike 5. Međusignali prate izlaze pojedinačnih kola.

```

module zadatak (
    input logic A, B, C, D,
    output logic Y
);
    logic A_n, B_n, C_n, D_n;
    logic A_p, B_p, C_p, D_p;
    logic I1, I2, I3, I4, I5;
    // Potpun ulazni razvod sa osam invertora, kao na semi.
    assign A_n = ~(A & A);
    assign A_p = ~(A_n & A_n);
    assign B_n = ~(B & B);
    assign B_p = ~(B_n & B_n);
    assign C_n = ~(C & C);
    assign C_p = ~(C_n & C_n);
    assign D_n = ~(D & D);
    assign D_p = ~(D_n & D_n);
    // Svaka dodela predstavlja jedno dvoulazno NI kolo.
    assign I1 = ~(A_n & D_p);
    assign I2 = ~(A_p & D_n);
    assign I3 = ~(I1 & I2);
    assign I4 = ~(C_n & I3);
    assign I5 = ~(B_p & C_p);
    assign Y = ~(I4 & I5);
endmodule

```

Testbench i automatske provere: [tb_zadatak.sv](#).

d) U okviru ove tačke zahtevana je implementacija funkcije (2.1) korišćenjem isključivo dvoulaznih NILI kola. Sistematski pristup za implementaciju funkcije koja sadrži isključivo dvoulazna NILI kola podrazumeva sledeća dva koraka:

- 1) Transformaciju minimalne funkcije u formi proizvoda zbirova u algebarski zapis koji sadrži isključivo komplementirane zbirove

- 2) Šemu koja sadrži isključivo NILI kola a koja se dobija na osnovu algebarskog izraza dobijenog u koraku 1

Korak (1) obuhvata niz transformacija koje za cilj imaju dobijanje algebarskog zapisa u kome isključivo figurišu komplementirani zbrovi sa tačno dva sabirka. Kao polazni izraz, nad kojim će se vršiti dalje transformacije, koristi se forma proizvoda zbrova funkcije dobijene primenom Karnoove karte. Za slučaj funkcije (2.1) polazni izraz predstavlja (2.3) ili ponovljeno ispod:

$$Y = (A + C + D)(\bar{A} + C + \bar{D})(B + \bar{C}) \quad (2.11)$$

Na osnovu primene pravila 7 Bulove algebre definisanog u Tabeli 2 moguće je transformisati (2.11) u:

$$Y = (C + (A + D)(\bar{A} + \bar{D}))(B + \bar{C}) \quad (2.12)$$

Dvostrukim komplementiranjem izraza (2.12) dobija se

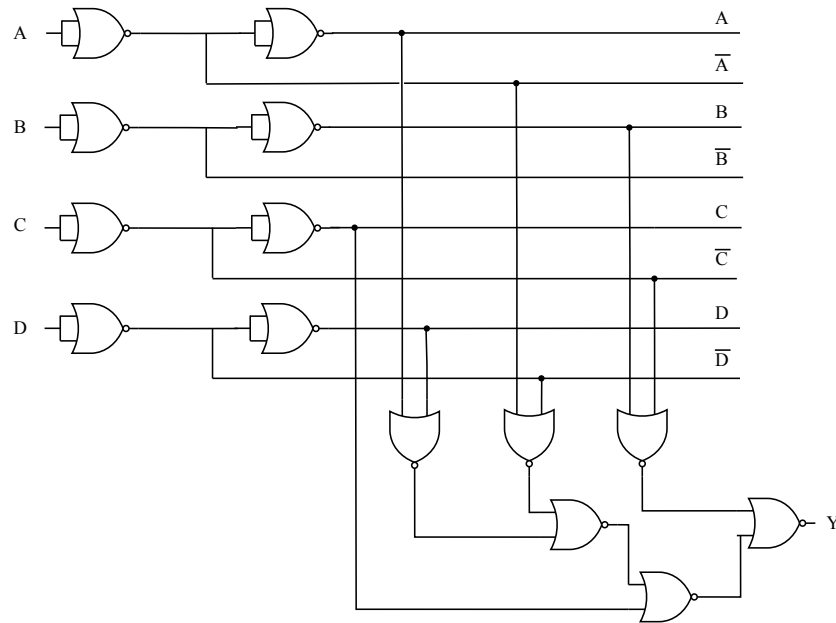
$$Y = \bar{\bar{Y}} = \overline{(C + (A + D)(\bar{A} + \bar{D}))(B + \bar{C})} \quad (2.13)$$

$$Y = \overline{(C + (A + D)(\bar{A} + \bar{D})) + (B + \bar{C})} \quad (2.14)$$

$$Y = C + (A + D)(\bar{A} + \bar{D}) + B + \bar{C} \quad (2.15)$$

$$Y = C + \bar{A} + D + \bar{A} + \bar{D} + B + \bar{C} \quad (2.16)$$

Na slici 6 je predstavljena implementacija funkcije (2.16).



Slika 6: Implementacija funkcije (2.1) korišćenjem isključivo dvoulaznih NILI kola.

Verilog kod: realizacija isključivo dvoulaznim NILI kolima sa slike 6. Inverzije se dobijaju vezivanjem oba ulaza istog kola.

```
module zadatak (
    input logic A, B, C, D,
    output logic Y
);
    logic A_n, B_n, C_n, D_n;
    logic A_p, B_p, C_p, D_p;
    logic I1, I2, I3, I4, I5;
    // Potpun ulazni razvod sa osam invertora, kao na semi.
    assign A_n = ~(A | A);
    assign A_p = ~(A_n | A_n);
    assign B_n = ~(B | B);
    assign B_p = ~(B_n | B_n);
    assign C_n = ~(C | C);
    assign C_p = ~(C_n | C_n);
    assign D_n = ~(D | D);
    assign D_p = ~(D_n | D_n);
    // Svaka dodela predstavlja jedno dvoulazno NILI kolo.
    assign I1 = ~(A_p | D_p);
    assign I2 = ~(A_n | D_n);
    assign I3 = ~(I1 | I2);
    assign I4 = ~(C_p | I3);
    assign I5 = ~(B_p | C_n);
    assign Y = ~(I4 | I5);
endmodule
```

Testbench i automatske provere: [tb_zadatak.sv](#).

Primeri su napisani u SystemVerilogu (fajlovi `.sv`) i simuliraju se Verilatorom.

Verilog kod koji implementira funkcije (2.2) i (2.3).

```
module zadatak (
    input logic A,
    input logic B,
    input logic C,
    input logic D,
    output logic Y_ZP,
    output logic Y_PZ
);
    logic A_n, C_n, D_n;
    // Invertovane vrednosti signala
    assign A_n = ~A;
    assign C_n = ~C;
    assign D_n = ~D;
    //  $Y_{ZP} = A_n C_n D + A C_n D_n + BC$ 
    assign Y_ZP = (A_n & C_n & D) | (A & C_n & D_n) | (B & C);
    //  $Y_{PZ} = (A + C + D)(A_n + C + D_n)(B + C_n)$ 
    assign Y_PZ = (A | C | D) & (A_n | C | D_n) & (B | C_n);
endmodule
```

Testbench koji simulira date funkcije je dat [OVDE](#).

2.2 Zadatak 2

- a) Odrediti funkcionalnu tabelu za funkciju $Z = f(A, B, C, D)$, gde je Z jednako 1 kada u zapisu četvorobitnog broja $ABCD$ postoje barem dve susedne nule, dok je u suprotnom Z jednako 0 (na primer, kada je $ABCD = 0000$ tada je $Z = 1$, dok kada je $ABCD = 0101$ tada je $Z = 0$).
- b) Projektovati kombinacionu mrežu koja realizuje funkciju Z ukoliko su dostupna logička kola proizvoljnog tipa.

Napomena. Težiti da broj upotrebljenih logičkih kola bude minimalan.

Rešenje:

- a) Sadržaj funkcionalne tabele za funkciju Z je:

Tabela 3: Funkcionalna tabela za $Z = f(A, B, C, D)$

A	B	C	D	Z
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

b) Pošto se postavkom zadatka zahteva da broj upotrebljenih kola bude minimalan, realizaciju ove logičke funkcije potrebno je započeti korišćenjem metode minimizacije bazirane na upotrebi Karnoovih karti. Sadržaj Karnoove karte za funkcionalnu tabelu 3 iz prethodne tačke je prikazan na slici 7.

Funkciju Z ćemo najpre izraziti u obe forme (zbir proizvoda i proizvod zbirova) a zatim ćemo analizom utvrditi koja forma zahteva manji broj logičkih kola.

Na osnovu slike 8 dobijamo sledeće funkcije:

$$Z_{MIN_ZP} = \bar{C}\bar{D} + \bar{A}\bar{B} + \bar{B}\bar{C} \quad (2.17)$$

$$Z_{MIN_PZ} = (\bar{B} + \bar{C})(\bar{B} + \bar{D})(\bar{A} + \bar{C}) \quad (2.18)$$

Obe prikazane dvonivojske forme zahtevaju po četiri glavna kola i četiri invertora. To poređenje ne dokazuje minimalnost među svim višenivojskim realizacijama. Na slici je, kao u zadatku 1,

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	1	0	0	0
10	1	1	0	0

Slika 7: Sadržaj Karnoove karte za funkciju Z .

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	1	0	0	0
10	1	1	0	0

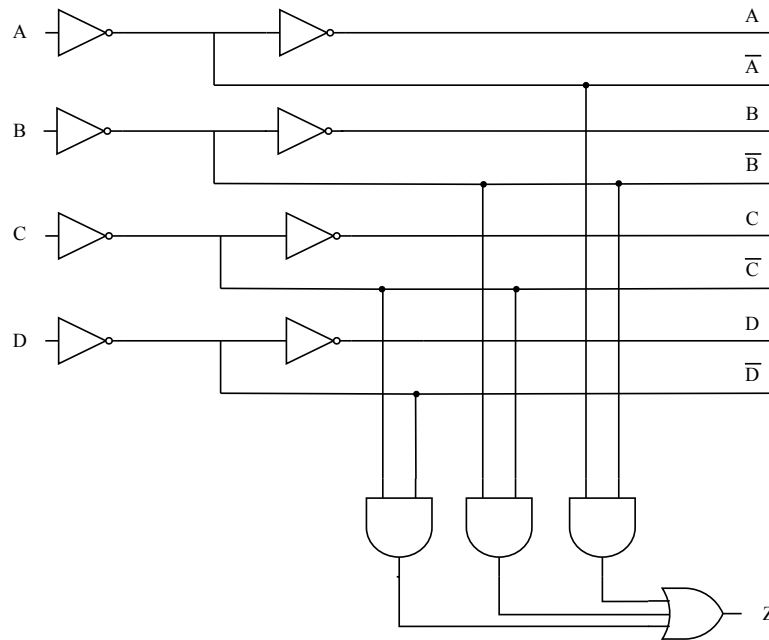
(a) Proizvod zbirova

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	1	0	0	0
10	1	1	0	0

(b) Zbir proizvoda**Slika 8:** Karnoova karta za funkciju Z gde je a) označena oblast za minimizaciju u formi proizvoda zbirova b) označena oblast za minimizaciju u formi zbira proizvoda.

prikazan potpun ulazni razvod sa osam invertora, pa prikazana šema sa razvodom ima dvanaest kola. Za realizaciju sa osam kola dovoljno je po jedno invertovanje svakog ulaza.

Na slici 9 je prikazana implementacija funkcije (2.17).



Slika 9: Implementacija funkcije Z korišćenjem forme zbira proizvoda.

Verilog kod koji implementira funkcije (2.17) i (2.18). U Verilog kodu portovi Y_ZP i Y_PZ predstavljaju izlaz Z u dve realizacije.

```
module zadatak (
    input logic A,
    input logic B,
    input logic C,
    input logic D,
    output logic Y_ZP,
    output logic Y_PZ
);
    logic A_n, B_n, C_n, D_n;
    // Invertovane vrednosti signala
    assign A_n = ~A;
    assign B_n = ~B;
    assign C_n = ~C;
    assign D_n = ~D;
    // Y_ZP = C_n D_n + A_n B_n + B_n C_n
    assign Y_ZP = (C_n & D_n) | (A_n & B_n) | (B_n & C_n);
    // Y_PZ = (B_n + C_n)(B_n + D_n)(A_n + C_n)
    assign Y_PZ = (B_n | C_n) & (B_n | D_n) & (A_n | C_n);
endmodule
```

Testbench koji simulira funkcije (2.17) i (2.18) je dat [OVDE](#).

2.3 Zadatak 3

Projektovati kombinacionu mrežu koja realizuje izlaz $C(C_3C_2C_1C_0) = A(A_1A_0) \cdot B(B_1B_0)$, gde su A i B neoznačeni binarni brojevi. Kombinacionu mrežu je potrebno realizovati korišćenjem minimalnog broja osnovnih logičkih kola sa proizvoljnim brojem ulaza.

Rešenje:

Prvi korak u rešavanju ovog zadatka predstavlja kreiranje sadržaja funkcionalne tabele:

Tabela 4: Funkcionalna tabela za $C = A \cdot B$

A_1	A_0	B_1	B_0	C_3	C_2	C_1	C_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

Nakon kreiranja funkcionalne tabele, primenom metode optimizacije bazirane na Karnoovim kartama, potrebno je realizovati minimalnu funkciju za svaki od izlaznih signala. Sadržaj Karnoovih karti je prikazan na slikama 10 i 11.

B_1B_0 A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	1	0
10	0	0	0	0

(a) Karnoova mapa za C_3

B_1B_0 A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	1
10	0	0	1	1

(b) Karnoova mapa za C_2

Slika 10: Karnoove karte za C_3 i C_2 proizvoda $C = A \cdot B$, sa grupisanim jedinicama (ZP).

$\begin{smallmatrix} B_1B_0 \\ A_1A_0 \end{smallmatrix}$	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	1	0	1
10	0	1	1	0

(a) Karnoova mapa za C_1

$\begin{smallmatrix} B_1B_0 \\ A_1A_0 \end{smallmatrix}$	00	01	11	10
00	0	0	0	0
01	0	1	1	0
11	0	1	1	0
10	0	0	0	0

(b) Karnoova mapa za C_0 **Slika 11:** Karnoove karte za C_1 i C_0 proizvoda $C = A \cdot B$, sa grupisanim jedinicama (ZP).

Na osnovu sadržaja Karnoovih karti, prikazanih na slikama 10 i 11, moguće je realizovati svaku od izlaznih funkcija:

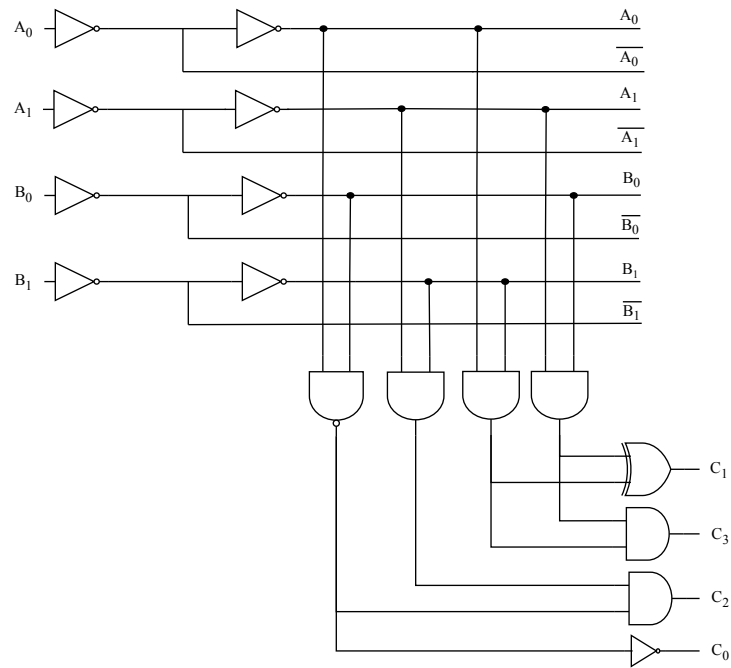
$$C_3 = A_1 A_0 B_1 B_0 \quad (2.19)$$

$$C_2 = A_1 B_1 \bar{B}_0 + \bar{A}_0 A_1 B_1 = A_1 B_1 (\bar{B}_0 + \bar{A}_0) = A_1 B_1 \overline{A_0 B_0} \quad (2.20)$$

$$\begin{aligned} C_1 &= A_1 \bar{B}_1 B_0 + A_1 \bar{A}_0 B_0 + \bar{A}_1 A_0 B_1 + A_0 B_1 \bar{B}_0 \\ &= A_1 B_0 (\bar{A}_0 + \bar{B}_1) + A_0 B_1 (\bar{A}_1 + \bar{B}_0) \\ &= A_1 B_0 (\overline{A_0 B_1}) + A_0 B_1 (\overline{A_1 B_0}) = A_1 B_0 \oplus A_0 B_1 \end{aligned} \quad (2.21)$$

$$C_0 = A_0 B_0 \quad (2.22)$$

Na slici 12 je predstavljena implementacija funkcije $C = A \cdot B$. Ulazni razvod daje komplement i direktni signal svakog ulaza; pošto se koriste samo direktni ulazi, njegovih osam invertora nije potrebno u minimalnoj realizaciji.

Slika 12: Implementacija funkcije $C = A \cdot B$.

Verilog kod koji implementira funkciju $C = A \cdot B$.

```

module zadatak (
    input logic [1:0] A,
    input logic [1:0] B,
    output logic [3:0] C
);
    // C0 = A0 B0
    assign C[0] = A[0] & B[0];
    // C1 = (A1 B0) xor (A0 B1)
    assign C[1] = (A[1] & B[0]) ^ (A[0] & B[1]);
    // C2 = (A1 B1) (A0 nand B0)
    assign C[2] = (A[1] & B[1]) & ~(A[0] & B[0]);
    // C3 = A1 A0 B1 B0
    assign C[3] = A[1] & A[0] & B[1] & B[0];
endmodule

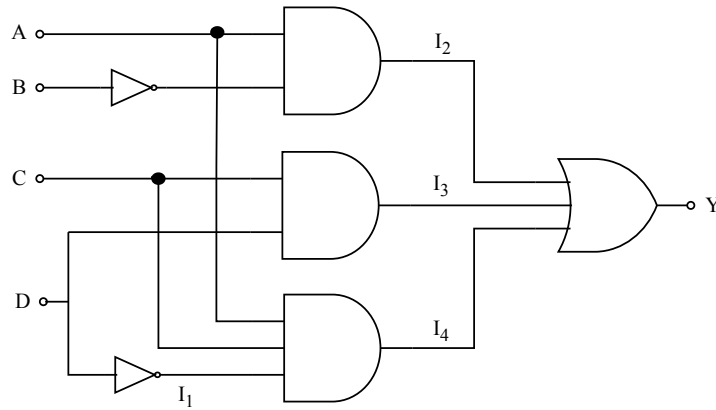
```

Testbench koji simulira funkciju $C = A \cdot B$ je dat [OVDE](#).

2.4 Zadatak 4

Za kolo predstavljeno na slici 13:

- Odrediti logičku funkciju
- Nacrtati vremenski dijagram prelaza ABCD: 1111 → 1110 za signale D , I_1 , I_2 , I_3 , I_4 i Y ako je kašnjenje svakog logičkog kola jednako T . Komentarisati dobijene vremenske dijagrame.
- Predložiti realizaciju koja rešava probleme uočene pod b)



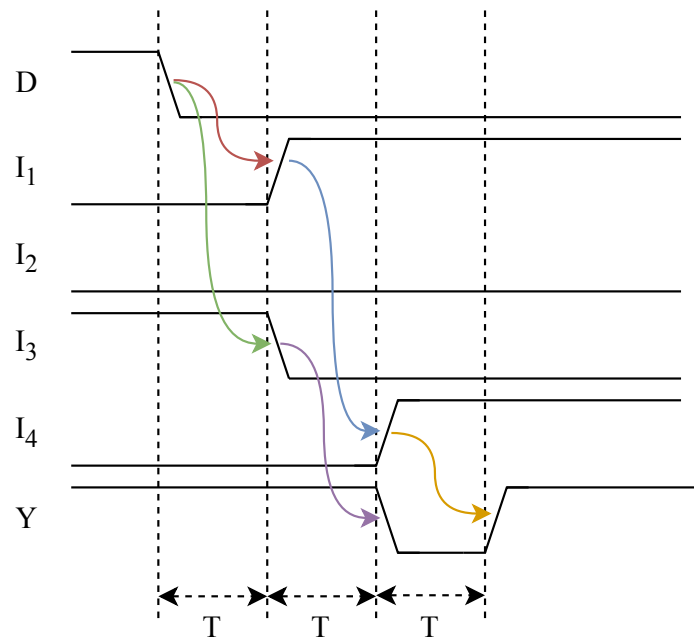
Slika 13: Šema logičkog kola uz zadatak 2.4.

Rešenje:

- Logička funkcija kojom se opisuje kolo prikazano na slici 13 je:

$$Y = A\bar{B} + CD + AC\bar{D} \quad (2.23)$$

- Vremenski dijagram prelaza ABCD: 1111 → 1110 je prikazan na slici 14.



Slika 14: Vremenski dijagram prelaza ABCD: 1111 → 1110 za logičko kolo sa slike 13.

Ako posmatramo vrednost funkcije definisane jednačinom (2.23) videćemo da za obe kombinacije ulaza $ABCD = 1111$ i $ABCD = 1110$ dobijamo istu vrednost. Međutim, uvidom u vremenski dijagram možemo uočiti da postoji jedan vremenski interval, jednak kašnjenju logičkog kola, u okviru koga vrednost izlaza Y postaje jednaka logičkoj nuli. Takvo ponašanje nije poželjno u logičkim kolima koja realizujemo jer se potencijalno može degradirati funkcionalnost ostatka digitalnog sistema koji koristi takvu realizaciju. Zbog toga se postavlja pitanje kako rešiti uočeni problem u ovom konkretnom slučaju i još mnogo bitnije pitanje, kako uopšteno detektovati da postoji problem u nekom kolu a zatim ga sistematično rešavati?

Na putu do odgovora na postavljena pitanja, potrebno je analizirati sadržaj Karnoove karte kao i odgovarajućih oblasti koje u toj Karnoovoj karti pokriva funkcija (2.23). Na slici 15 je predstavljen sadržaj Karnoove karte sa naznačenim odgovarajućim oblastima:

$\begin{array}{c} CD \\ AB \end{array}$	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	1
10	1	1	1	1

Slika 15: Sadržaj Karnoove karte za funkciju (2.23).

Polja 1111 i 1110 imaju izlaznu jedinicu, ali ih nijedan isti proizvod u realizaciji (2.23) ne pokriva. Pri promeni D jedan proizvod može prestati da vodi pre nego što drugi počne da vodi, pa se javlja lažna nula. Za dvonivojsku realizaciju ZP proverava se zajedničko pokrivanje svakog para susednih jedinica, a za PZ svakog para susednih nula. Sama blizina ili preklapanje kontura nije dovoljan kriterijum. Mogućnost impulsa razmatra se za oba smera prelaza; njegova stvarna pojava zavisi od modela i odnosa kašnjenja.

Za samostalni rad: Nacrtati vremenske dijagrame za prelaz $ABCD: 1110 \rightarrow 1111$.

c) Problem rešavamo dodavanjem ili proširivanjem člana koji zajednički pokriva oba kraja opasnog prelaza. Za slučaj u ovom zadatku dobijamo oblasti kao što je prikazano na slici 16.

Na osnovu Karnoove karte sa slike 16, izvodimo sledeću funkciju:

$$Y = A\bar{B} + CD + AC \quad (2.24)$$

Za samostalni rad: Nacrtati vremenske dijagrame za funkciju (2.24) za prelaz $ABCD: 1110 \rightarrow 1111$.

Poređenjem funkcija (2.24) i (2.23) možemo zaključiti da je primena metode rešavanja statičkog hazarda (pojava lažne nule ili lažne jedinice) u ovom konkretnom slučaju doprinela i dodatnoj minimizaciji logičke funkcije (2.23) što **NE MORA** uvek biti slučaj.

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	1
10	1	1	1	1

Slika 16: Karnoova karta za funkciju (2.23).

Verilog kod koji implementira funkciju (2.23) sa kašnjenjem T .

Napomena: Kašnjenja služe samo simulaciji. Zapis `assign #(T)` podrazumeva inertno kašnjenje: kraći impulsi mogu biti potisnuti. Transportni model prenosi i takve impulse; prikazani primer koristi inertni model. Za zadati prelaz iz zadatka 4 ulazni impuls izlaznog kola traje tačno T ; prikazani izlazni impuls traje od $2T$ do $3T$ posle promene D . Odsustvo impulsa u jednoj simulaciji nije opšti dokaz odsustva hazarda.

```

module zadatak #(parameter time T = 1ns) (
    input logic A,
    input logic B,
    input logic C,
    input logic D,
    output logic Y
);
    logic B_inv, I1;
    logic I2, I3, I4;
    // B_inv = inv B, I1 = inv D
    assign #(T) B_inv = (~B);
    assign #(T) I1 = (~D);
    // I2 = A B_inv
    assign #(T) I2 = (A & B_inv);
    // I3 = C D
    assign #(T) I3 = (C & D);
    // I4 = A and C and I1 (A C \bar{D})
    assign #(T) I4 = (A & C & I1);
    // Y = I2 + I3 + I4
    assign #(T) Y = (I2 | I3 | I4);
endmodule

```

Testbench koji simulira prelazak $ABCD = 1111 \rightarrow ABCD = 1110$ je dat OVDE.

Verilog kod: proširena realizacija jednačine (2.24). Član AC ostaje aktivan tokom oba smera promene D kada je $A = C = 1$. Svaka dodela ima inertno kašnjenje T . Podrazumevano $T = 1$ znači 1 ns prema podešavanju simulatora, bez vremenskih deklaracija u RTL-u. Testbench poredi ovu mrežu sa mrežom koja ima hazard, uz $T = 1$ ns i $T = 7$ ns.

```
module zadatak #(parameter time T = 1) (
    input logic A, B, C, D,
    output logic Y
);
    logic B_inv, I2, I3, I4;
    assign #(T) B_inv = ~B;
    assign #(T) I2 = A & B_inv;
    assign #(T) I3 = C & D;
    // Član AC pokriva oba kraja prelaza D: 1 <-> 0.
    assign #(T) I4 = A & C;
    assign #(T) Y = I2 | I3 | I4;
endmodule
```

Pomoćni moduli: [sa_hazardom.sv](#).

Testbench i automatske provere: [tb_zadatak.sv](#).

2.5 Zadatak 5

Data je funkcija

$$Y = \bar{A}\bar{C} + CD + BCD\bar{C} \quad (2.25)$$

- Odrediti pri kojim prelazima postoji mogućnost generisanja lažnih nula na izlazu kola.
- Predstaviti funkciju (2.25) u minimalnoj formi
- Ukoliko u realizaciji b) postoji mogućnost generisanja lažnih nula, modifikovati funkciju tako da se to spreči

Rešenje:

Na slici 17 je prikazan sadržaj Karnoove karte za funkciju (2.25).

CD \ AB	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	1	1
10	0	0	1	0

Slika 17: Sadržaj Karnoove karte za funkciju (2.25).

Na osnovu obeleženih kontura koje odgovaraju pojedinačnim logičkim proizvodima iz logičke funkcije (2.25), vidimo da funkcija nije data u minimalnoj formi. Takođe, uočavamo potencijalne prelaze koji mogu dovesti do generisanja lažne nule na izlazu kola. To su prelazi između susednih polja sa jedinicom koja nisu pokrivena nijednim zajedničkim proizvodom, pri promeni tačno jednog ulaznog signala. Na slici 17 su ovi prelazi označeni parom strelica na levu i desnu stranu.

Prema tabeli sa slike 17 uočavamo da postoji ukupno 10 prelaza koji mogu dovesti do pojave lažne nule na izlazu kola. Kao što je obeleženo u Karnoovoj karti na slici 17, polja $ABCD = 0100$ i $ABCD = 0110$ predstavljaju susedna polja u Karnoovoj karti.

b) Na slici 18 je predstavljen sadržaj Karnoove karte sa označenim oblastima za slučaj minimalne forme funkcije (2.25). Kao što se sa slike 18 može primetiti, minimalna forma funkcije i dalje sadrži prelaze koji potencijalno mogu uzrokovati pojavu lažnih nula.

$\begin{array}{c} CD \\ \backslash AB \end{array}$	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	1	1
10	0	0	1	0

Slika 18: Sadržaj Karnoove karte za slučaj minimalne forme funkcije (2.25) sa označenim prelazima koji mogu uzrokovati pojavu lažnih nula.

Na osnovu slike 18 dobijamo funkciju (2.25) u minimalnoj formi:

$$Y = \bar{A}\bar{C} + CD + BC \quad (2.26)$$

c) Na slici 19 je prikazana Karnoova karta sa naznačenim oblastima koje omogućavaju realizaciju funkcije u formi koja neće dovesti do generisanja lažnih nula.

$\begin{array}{c} CD \\ \backslash AB \end{array}$	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	0	1	1
10	0	0	1	0

Slika 19: Karnoova karta sa naznačenim oblastima koje omogućavaju realizaciju funkcije (2.25) u formi koja neće dovesti do generisanja lažnih nula.

Na osnovu Karnoove karte, prikazane na slici 19, moguće je izvesti sledeći zapis funkcije:

$$Y = \bar{A}\bar{C} + CD + BC + \bar{A}B + \bar{A}D \quad (2.27)$$

Dakle, polazeći od funkcije (2.25) najpre smo u tački b) izveli izraz koji predstavlja funkciju (2.25) u minimalnoj formi (2.26). Međutim tako dobijena forma je i dalje potencijalno generisala lažne nule. Da bismo sprečili generisanje lažnih nula, na osnovu preklapanja oblasti u

Karnoovoj karti dobili smo formu (2.27). Za razliku od (2.26), forma predstavljena u okviru (2.27) sadrži veći broj logičkih kola ali sprečava pojavu lažnih nula.

Verilog kod koji implementira funkcije (2.26) i (2.27) sa kašnjenjem T.

Napomena: Kašnjenja služe samo simulaciji; `assign #(T)` koristi inertni model, objašnjen uz zadatak 2.4.

```
module zadatak #(parameter time T = 1ns) (
    input logic A,
    input logic B,
    input logic C,
    input logic D,
    output logic Y_min,
    output logic Y_no_hazard
);
    logic A_inv, C_inv;
    logic I1, I2, I3, I4, I5;
    // Invertovane vrednosti
    assign #(T) A_inv = (~A);
    assign #(T) C_inv = (~C);
    // I1 = A_n C_n
    assign #(T) I1 = (A_inv & C_inv);
    // I2 = C D
    assign #(T) I2 = (C & D);
    // I3 = B C
    assign #(T) I3 = (B & C);
    // I4 = A_n B
    assign #(T) I4 = (A_inv & B);
    // I5 = A_n D
    assign #(T) I5 = (A_inv & D);
    // Y_min = A_n C_n + C D + BC = I1 + I2 + I3
    assign #(T) Y_min = (I1 | I2 | I3);
    // Y_no_hazard = A_n C_n + CD + BC + A_nB + A_nD
    // = I1 + I2 + I3 + I4 + I5
    assign #(T) Y_no_hazard = (I1 | I2 | I3 | I4 | I5);
endmodule
```

Testbench koji simulira funkcije (2.26) i (2.27) je dat [OVDE](#).

2.6 Zadatak 6

Data je funkcija u formi zbira logičkih proizvoda:

$$Y = \bar{B}\bar{C}\bar{D} + \bar{A}\bar{C}D + \bar{A}C\bar{D} \quad (2.28)$$

- Logičku funkciju (2.28) predstaviti u formi proizvoda logičkih zbirova. Da li, i pri kojim prelazima, postoji mogućnost generisanja lažnih jedinica na izlazu kola?
- Modifikovati logičku funkciju tako da ne postoji mogućnost generisanja lažnih jedinica

Rešenje:

- Na slici 20 je predstavljen sadržaj Karnoove karte dobijen na osnovu funkcije (2.28).

$\begin{array}{c} CD \\ AB \end{array}$	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	1	0	0
10	1	1	0	0

Slika 20: Sadržaj Karnoove karte za funkciju (2.28).

Na osnovu Karnoove karte prikazane na slici 20, određujemo logičku funkciju izlaza kola u minimalnoj formi u obliku proizvoda logičkih zbirova:

$$Y = (\bar{B} + C + D)(A + \bar{D})(\bar{A} + \bar{C}) \quad (2.29)$$

Sa slike 20 uočavamo da logička funkcija (2.29) u traženom obliku ima četiri para susednih nula bez zajedničkog pokrivanja, odnosno osam usmerenih prelaza sa mogućnošću pojave lažne jedinice. To su parovi (3, 11), (4, 5), (7, 15) i (12, 14), gde brojevi označavaju dekadne vrednosti ulaza $ABCD$. Na slici 21 obeleženi su svi takvi prelazi.

$\begin{array}{c} CD \\ AB \end{array}$	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	1	0	0
10	1	1	0	0

Slika 21: Karnoova karta funkcije (2.28) sa naznačenim prelazima koji dovode do pojave lažne jedinice.

$\begin{smallmatrix} CD \\ AB \end{smallmatrix}$	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	0	1	0	0
10	1	1	0	0

Slika 22: Karnoova karta funkcije (2.28) sa dodatnim oblastima koje zajednički pokrivaju susedne nule i sprečavaju statički hazard 0 pri promeni jednog ulaza.

b) Nova forma funkcije (2.28) koja ne dovodi do pojave lažnih jedinica dobija se na osnovu Karnoove karte prikazane na slici 22.

Na osnovu Karnoove karte prikazane na slici 22 izvodimo sledeću funkciju:

$$Y = (\bar{B} + C + D)(A + \bar{D})(\bar{A} + \bar{C})(\bar{C} + \bar{D})(A + \bar{B} + C)(\bar{A} + \bar{B} + D) \quad (2.30)$$

Verilog kod koji implementira funkcije (2.29) i (2.30) sa kašnjenjem T.

Napomena: Kašnjenja služe samo simulaciji; `assign #(T)` koristi inertni model, objašnjen uz zadatak 2.4.

```
module zadatak #(parameter time T = 1ns) (
    input logic A,
    input logic B,
    input logic C,
    input logic D,
    output logic Y_min,
    output logic Y_no_hazard
);
    logic A_inv, B_inv, C_inv, D_inv;
    logic I1, I2, I3, I4, I5, I6;
    // Invertovane vrednosti
    assign #(T) A_inv = (~A);
    assign #(T) B_inv = (~B);
    assign #(T) C_inv = (~C);
    assign #(T) D_inv = (~D);
    // I1 = (B_n + C + D)
    assign #(T) I1 = (B_inv | C | D);
    // I2 = (A + D_n)
    assign #(T) I2 = (A | D_inv);
    // I3 = (A_n + C_n)
    assign #(T) I3 = (A_inv | C_inv);
    // I4 = (C_n + D_n)
    assign #(T) I4 = (C_inv | D_inv);
    // I5 = (A + B_n + C)
    assign #(T) I5 = (A | B_inv | C);
    // I6 = (A_n + B_n + D)
    assign #(T) I6 = (A_inv | B_inv | D);
    // Y_min = (B_n + C + D)(A + D_n)(A_n + C_n) = I1 I2 I3
    assign #(T) Y_min = (I1 & I2 & I3);
    // Y_no_hazard =
    // (B_n + C + D)(A + D_n)(A_n + C_n)(C_n + D_n)(A + B_n + C)(A_n + B_n + D)
    // I1 I2 I3 I4 I5 I6
    assign #(T) Y_no_hazard = (I1 & I2 & I3 & I4 & I5 & I6);
endmodule
```

Testbench koji simulira funkcije (2.29) i (2.30) je dat [OVDE](#).

3 Zadaci za samostalni rad

3.1 Zadatak 1

Projektovati kombinacionu mrežu koja na ulazima prihvata dva dvobitna neoznačena binarna broja, a na izlazima generiše binarni broj koji predstavlja zbir kvadrata ulaznih brojeva. Na raspolaganju su proizvoljna logička kola niskog stepena integracije. Težiti da broj upotrebljenih logičkih kola bude minimalan.

3.2 Zadatak 2

Projektovati kombinacionu mrežu koja za četvorobitni binarni broj na ulazu generiše binarni broj koji predstavlja broj pojavljivanja jedinice u ulaznom binarnom broju. Na raspolaganju su proizvoljna logička kola. Težiti da broj upotrebljenih logičkih kola bude minimalan.